model_inverses

October 22, 2026

1 Model Inverses

The multivector inverse models here are written for design clarity. The production versions employ numpy and numba for execution speed; making the production versions harder to read.

It is important to realize that the multivectors here are in *ordinal* order, so that the binary index for the multivector reveals the Grassman identity. For instance, the multivector element associated with e_{236} is stored in A.Reg[0b100110]. This is especially important for involution templates. The publicly available `clifford` Python package stores multivector elements differently. Luckily the scalar and pseudoscalar elements are in the same places (first and last).

1.1 About the algorithms

The even dimensions are simplest. The odd dimensions require composite scalar/pseudoscalar processing: both in the tweak, and in the "determinant". Odd cases derive FLS processing order from the preceeding even case. Think of the odd cases as extending the preceeding even case.

The last step to completing the general pattern was finding the right involutions. Reversion sits at the base of every case but an additional involution is composed on top of the reversion. Those reversion modifications are described below, and they follow a predictable pattern.
- When we step from an odd dimension to an even one, we simply double the size of each piece of the odd base. - When we step from an even dimension to an odd one, we concatenate the complement of the even base. The pattern to the odd contribution is termed "Thue-Morse". This is the pattern found in the pseudoscalar sign table, or the last row (or column) of Sylvester's Hadamard matrix.

[1]:
```
import numpy as np
import clif.util as util
import clif.context as Clif
from clif.multivector import Accum

VERBOSE = True
TOLERANCE = 1.0e-12
# A is a multivector and rA is its inverse
# test 1 == A*rA ? and refine the inverse if out of tolerance
def Newton_Schulz(A: Accum, rA: Accum ) -> Accum:
    max_was = 1.0
    while True:
        # evaluate
        E = A * rA
```

```
        max = np.abs(E.Reg[0]-1.0)
        for i in range(1,E.bases):
            err = np.abs(E.Reg[i])
            if err > max:
                max = err
        if VERBOSE: print(f"residual < {max}")

        if max < TOLERANCE:
            return rA
        elif max > max_was:
            print("reciprocal refinement diverges")
            return None
        else:
            max_was = max
            rA *= 2 - E
```

[2]:
```
# Lounesto names for common involutions
#reversion = [[1,1,-1,-1][3 & Clif.Grade[i]] for i in range(1<<dimensions)]
#conjugation = [[1,-1,-1,1][3 & Clif.Grade[i]] for i in range(1<<dimensions)]
#grade_involution = [[1,-1][1 & Clif.Grade[i]] for i in range(1<<dimensions)]

invo_4   = [ 1]*16
invo_5   = [ 1]*32
invo_6   = [ 1]*64
invo_7   = [ 1]*64  + [-1]*64
invo_8   = [ 1]*128 + [-1]*128
invo_9   = [ 1]*128 + [-1]*128 + [-1]*128 + [ 1]*128
invo_10  = [ 1]*256 + [-1]*256 + [-1]*256 + [ 1]*256
invo_11  = [ 1]*256 + [-1]*256 + [-1]*256 + [ 1]*256 + [-1]*256 + [ 1]*256 + [␣
 ↪1]*256 + [-1]*256
invo_12  = [ 1]*512 + [-1]*512 + [-1]*512 + [ 1]*512 + [-1]*512 + [ 1]*512 + [␣
 ↪1]*512 + [-1]*512
invo_13  = [ 1]*512 + [-1]*512 + [-1]*512 + [ 1]*512 + [-1]*512 + [ 1]*512 + [␣
 ↪1]*512 + [-1]*512
invo_13 += [-1]*512 + [ 1]*512 + [ 1]*512 + [-1]*512 + [ 1]*512 + [-1]*512 +␣
 ↪[-1]*512 + [ 1]*512
invo_14  = [ 1]*1024 + [-1]*1024 + [-1]*1024 + [ 1]*1024 + [-1]*1024 + [␣
 ↪1]*1024 + [ 1]*1024 + [-1]*1024
invo_14 += [-1]*1024 + [ 1]*1024 + [ 1]*1024 + [-1]*1024 + [ 1]*1024 +␣
 ↪[-1]*1024 + [-1]*1024 + [ 1]*1024
invo_15  = [ 1]*1024 + [-1]*1024 + [-1]*1024 + [ 1]*1024 + [-1]*1024 + [␣
 ↪1]*1024 + [ 1]*1024 + [-1]*1024
invo_15 += [-1]*1024 + [ 1]*1024 + [ 1]*1024 + [-1]*1024 + [ 1]*1024 +␣
 ↪[-1]*1024 + [-1]*1024 + [ 1]*1024
```

```
invo_15 += [-1]*1024 + [ 1]*1024 + [ 1]*1024 + [-1]*1024 + [ 1]*1024 +␣
 ↪[-1]*1024 + [-1]*1024 + [ 1]*1024
invo_15 += [ 1]*1024 + [-1]*1024 + [-1]*1024 + [ 1]*1024 + [-1]*1024 + [␣
 ↪1]*1024 + [ 1]*1024 + [-1]*1024

involution_fix = (invo_4, invo_5, invo_6, invo_7, invo_8, invo_9, invo_10,␣
 ↪invo_11, invo_12, invo_13, invo_14, invo_15)

reverse_pattern = [1,1,-1,-1]
def inverse_involution(dimensions):
    if dimensions < 4 or dimensions > 13:
        print("use another method")
        return None
    fix = involution_fix[dimensions-4]
    return [[1,1,-1,-1][3 & Clif.Grade[n]] * fix[n] for n in␣
 ↪range(1<<dimensions)]

tweak_4_5   = [1]
tweak_6_7   = [3, 1, 1/3]
tweak_8_9   = [7, 3, 5/3, 1, 3/5, 1/3, 1/7]
tweak_10_11 = [15, 7, 13/3, 3, 11/5, 5/3, 9/7, 1, 7/9, 3/5, 5/11, 1/3, 3/13, 1/
 ↪7, 1/15]
tweak_12_13 = [31, 15, 29/3, 7, 27/5, 13/3, 25/7, 3, 23/9, 11/5, 21/11, 5/3, 19/
 ↪13, 9/7, 17/15, 1,
               15/17, 7/9, 13/19, 3/5, 11/21, 5/11, 9/23, 1/3, 7/25, 3/13, 5/27,␣
 ↪1/7, 3/29, 1/15, 1/31]
tweak_14_15 = [63/1,62/2,61/3,60/4,59/5,58/6,57/7,56/8,55/9,54/10,53/11,52/
 ↪12,51/13,50/14,49/15,48/16,
                47/17,46/18,45/19,44/20,43/21,42/22,41/23,40/24,39/25,38/26,37/
 ↪27,36/28,35/29,34/30,33/31,
                32/32,31/33,30/34,29/35,28/36,27/37,26/38,25/39,24/40,23/41,22/
 ↪42,21/43,20/44,19/45,18/46,
                17/47,16/48,15/49,14/50,13/51,12/52,11/53,10/54,9/55,8/56,7/57,6/
 ↪58,5/59,4/60,3/61,2/62,1/63]

tweak = [         #     FLS  N
    None,         #  0
    None,         #  1
    None,         #  2
    None,         #  3
    tweak_4_5,    #  4   1   2
    tweak_4_5,    #  5   2   2
    tweak_6_7,    #  6   3   4
    tweak_6_7,    #  7   4   4
    tweak_8_9,    #  8   5   8
    tweak_8_9,    #  9   6   8
```

```
    tweak_10_11, # 10   7  16
    tweak_10_11, # 11   8  16
    tweak_12_13, # 12   9  32
    tweak_12_13, # 13  10  32
    tweak_14_15, # 14  11  64
    tweak_14_15, # 15  12  64
]
```

[3]:
```
# the general form of the tweak array is
def make_tweak(n):
    m = 1<<n
    t = []
    for k in range(1,m):
        t += [f"{m-k}/{k}"]
    return t
for i in range(1,6):
    print(i,make_tweak(i))
```

```
1 ['1/1']
2 ['3/1', '2/2', '1/3']
3 ['7/1', '6/2', '5/3', '4/4', '3/5', '2/6', '1/7']
4 ['15/1', '14/2', '13/3', '12/4', '11/5', '10/6', '9/7', '8/8', '7/9', '6/10',
'5/11', '4/12', '3/13', '2/14', '1/15']
5 ['31/1', '30/2', '29/3', '28/4', '27/5', '26/6', '25/7', '24/8', '23/9',
'22/10', '21/11', '20/12', '19/13', '18/14', '17/15', '16/16', '15/17', '14/18',
'13/19', '12/20', '11/21', '10/22', '9/23', '8/24', '7/25', '6/26', '5/27',
'4/28', '3/29', '2/30', '1/31']
```

[4]:
```
def even_inverse( T, I, A ): # T = tweaks, I = involution template, A =␣
 ↪multivector
    B = I * A
    C = A * B # C = AB -> 1/A = B/C
    D = C.copy()
    D.Reg[0] *= -T[0]
    for i in range(1,len(T)):
        D *= C
        D.Reg[0] *= -T[i]
    E = D * C # scalar: E = DC -> 1/C = D/E -> 1/A = BD/E
    rC = (1/E.Reg[0]) * D
    rC = Newton_Schulz(C,rC)
    if rC == None: return None
    rA = B * rC
    rA = Newton_Schulz(A,rA)
#    if VERBOSE:
#        print('\nValidate that we have found rC = 1/C, by printing C * rC' )
#        print( C * rC )
#        print('\nValidate that we have found rA = 1/A, by printing A * rA' )
```

```
#         print( A * rA )
    return rA
```

[5]:
```
# This is the "tweak version" of the generalized 7D inverse copied from "A&S 7D␣
 ↪Inverse.ipynb" in /Clifford/.
# This is a model written for clarity of concept without regard to efficiency.

# T is the tweak list
# I is {-1,1} len(I)==len(A) describing the involution needed
def odd_inverse( T, I, A ):
    PSEUDO = (1<<A.dimensions)-1
    B = I * A
    C = A * B # C = AB -> 1/A = B/C
    D = C.copy()
    D.Reg[0] *= -T[0]
    D.Reg[PSEUDO] *= -T[0]
    for i in range(1,len(T)):
        D *= C
        D.Reg[0] *= -T[i]
        D.Reg[PSEUDO] *= -T[i]
    E = D * C # E[0,PSEUDO] are only non-zero elements
    # E=DC -> 1/C = D/E = E'D/E'E
    Ep = E.copy()
    Ep.Reg[PSEUDO] *= -1
    EpE = Ep * E # scalar only
    rC = (1 / EpE.Reg[0]) * Ep * D
    rC = Newton_Schulz(C,rC)
    if rC == None: return None
    rA = B * rC # 1/A = B/C
    rA = Newton_Schulz(A,rA)
#    if VERBOSE:
#        print('Validate that we have found rC = 1/C, by printing C * rC' )
#        print( C * rC )
#        print('Validate that we have found rA = 1/A, by printing A * rA' )
#        print( A * rA )
    return rA
```

[6]:
```
Dimensions = [4, 5, 6, 7, 8, 9, 10, 11, 12, 13]
for dimension in Dimensions:
    print(f"Dimension = {dimension}")
    Clif.Cl(dimension)
    A = Accum.random()
    inverse = odd_inverse if 1&dimension else even_inverse
    I = inverse( tweak[dimension], inverse_involution(dimension), A )
```

```
Dimension = 4
residual < 4.440892098500626e-16
residual < 5.204170427930421e-16
```

```
Dimension = 5
residual < 1.3877787807814457e-15
residual < 1.6479873021779667e-15
Dimension = 6
residual < 4.628242233906121e-15
residual < 4.447830992404533e-15
Dimension = 7
residual < 4.926614671774132e-15
residual < 5.270523600886534e-15
Dimension = 8
residual < 1.1081136008783687e-12
residual < 2.650657471292561e-15
residual < 6.58501031480796e-15
Dimension = 9
residual < 6.654827985093141e-13
residual < 6.656532963722483e-13
Dimension = 10
residual < 1.3146795102577757e-08
residual < 5.467827886175745e-15
residual < 7.011752289898254e-15
Dimension = 11
residual < 1.3074138578470111e-08
residual < 3.4590386110977533e-15
residual < 6.024694632067451e-15
Dimension = 12
residual < 0.001472596049501938
residual < 3.938546961106213e-05
residual < 2.8334828618388295e-08
residual < 1.5505660281463205e-14
residual < 1.639075160320136e-14
Dimension = 13
residual < 0.005519714234171441
residual < 0.0012192774956499971
residual < 1.6359619400531378e-05
residual < 3.912702026959147e-09
residual < 3.764377556369211e-15
residual < 5.864883231843088e-15
```